

Private Notebook

Asystent prawny RAG — dokumentacja projektu

Typ projektu: Aplikacja webowa (FastAPI + RAG)

Język implementacji: Python 3.10+

Modele LLM: Lokalne — przez Ollama

Baza wektorowa: ChromaDB (persistent)

Interfejs: Przeglądarka — Single Page App

Wymagana sieć: Brak — 100% offline

1. Opis systemu

Private Notebook to lokalna aplikacja webowa realizująca paradygmat Retrieval-Augmented Generation (RAG). Umożliwia użytkownikowi tworzenie osobistych notatników, wgrywanie dokumentów prawnych (PDF, DOCX, TXT) oraz prowadzenie wieloturuowych rozmow z asystentem AI opartym wyłącznie na lokalnych modelach językowych. Zadane dane nie opuszczają komputera użytkownika.

Główne funkcje

- Zarządzanie notatnikami (tworzenie, zmiana nazwy, usuwanie).
- Wgrywanie

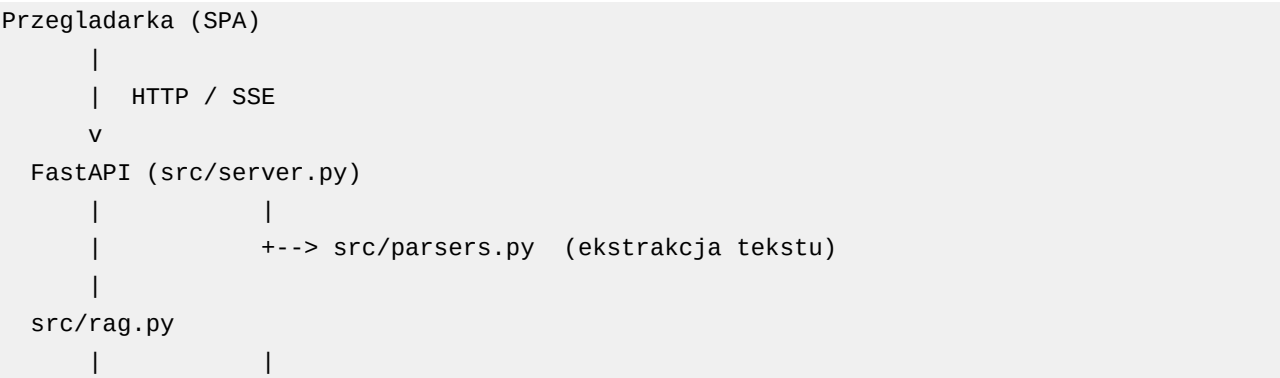
Wymagania systemowe

Python	>= 3.10
Ollama	>= 0.3 (lokalny serwer LLM)
RAM	>= 8 GB (16 GB zalecane dla modeli 7B)
Dysk	>= 5 GB wolnego miejsca na modele
System	Windows / macOS / Linux

2. Architektura

System składa się z warstwy backendowej (FastAPI), silnika RAG, parsera dokumentów i frontendu (SPA w czystym HTML/JS). Komunikacja z modelami językowymi odbywa się przez lokalne API serwera Ollama (<http://localhost:11434>).

Diagram komponentów



```
|          +--> ChromaDB  (wektory, trwały dysk)
|          |
|          +--> Ollama API  (embedding + generacja)
|
chats.json    (historia rozmow)
notebooks.json (metadane notatnikow)
```

Przepływ przetwarzania dokumentu

- Użytkownik wgrywa plik przez endpoint POST /api/upload.

• parsers.

Przepływ zapytania (chat)

- Użytkownik wysyła pytanie przez POST /api/chat.

• Pytanie

3. Struktura plików

```
private-notebook/
|
+-- doc/
|   +-- dokumentacja.pdf      <- niniejszy dokument
|
+-- src/
|   +-- __init__.py
|   +-- server.py             <- FastAPI app, endpointy REST/SSE
|   +-- rag.py                <- silnik RAG: chunking, embed, retrieve, chat
|   +-- parsers.py            <- parsery PDF / DOCX / TXT
|   +-- mock_ollama.py        <- lokalny mock Ollama do testow
|   +-- test_e2e.py           <- testy end-to-end (ASGI transport)
|
+-- static/
|   +-- index.html            <- SPA frontend
|
+-- requirements.txt
+-- run.bat                   <- skrypt startowy Windows
+-- run.sh                     <- skrypt startowy Linux/macOS
+-- .gitignore
+-- README.md
+-- LICENSE
```

Kluczowe moduły

src/server.py	Warstwa HTTP. Definiuje wszystkie endpointy API i serwuje frontend.
src/rag.py	Rdzen systemu. Chunking, embedding, wyszukiwanie wektorowe, generacja odpowiedzi.
src/parsers.py	Ekstrakcja tekstu z plikow PDF, DOCX i tekstowych.
src/mock_ollama.py	Serwer FastAPI imitujacy Ollama — uzywany w testach bez GPU.
src/test_e2e.py	Testy integracyjne via httpx.ASGITransport (bez portu sieciowego).
static/index.html	Interfejs uzytkownika — vanilla JS, dark mode, SSE streaming.

4. Instalacja i uruchamianie

Windows

```
# 1. Zainstaluj Ollama: https://ollama.com/download
# 2. Pobierz wymagane modele:
ollama pull llama3.2:3b
ollama pull nomic-embed-text

# 3. Uruchom aplikacje (tworzy venv i instaluje zaleznosci automatycznie):
run.bat
```

Linux / macOS

```
# 1. Zainstaluj Ollama: https://ollama.com/download
ollama pull llama3.2:3b && ollama pull nomic-embed-text

# 2. Uruchom aplikacje:
bash run.sh
```

Uruchamianie ręczne

```
python -m venv .venv
.venv\Scripts\activate          # Windows
# source .venv/bin/activate     # Linux/macOS
pip install -r requirements.txt
uvicorn src.server:app --port 8000 --reload
```

Aplikacja będzie dostępna pod adresem: <http://localhost:8000>

5. Dokumentacja API

Metoda	Sciezka	Opis
GET	/	Zwraca frontend (static/index.html)
GET	/api/health	Status Ollama i zaladowanych modeli
GET	/api/models	Lista dostepnych modeli LLM z flaga installed
POST	/api/pull	Strumieniuje postep pobierania modelu (SSE)
GET	/api/notebooks	Lista notatnikow (z liczba doc i czatow)
POST	/api/notebooks	Tworzy notatnik (body: {name})
PATCH	/api/notebooks/{id}	Zmienia nazwe notatnika (body: {name})
DELETE	/api/notebooks/{id}	Usuwa notatnik razem z dokumentami i czatami
GET	/api/documents	Lista dokumentow (parametr: ?notebook_id=)
POST	/api/upload	Wgrywa i indeksuje plik (form: file, notebook_id)
DELETE	/api/documents/{id}	Usuwa dokument i jego fragmenty wektorowe
POST	/api/reset	Czysci dokumenty (body: {notebook_id=?})
GET	/api/conversations	Lista konwersacji (parametr: ?notebook_id=)
POST	/api/conversations	Tworzy konwersacje (body: {model?, notebook_id})
GET	/api/conversations/{id}	Pobiera pelna konwersacje z historia wiadomosci
DELETE	/api/conversations/{id}	Usuwa konwersacje
POST	/api/chat	Strumieniuje odpowiedz (SSE): sources → token* → done

Format SSE (POST /api/chat)

```

event: sources
data:
[{"n":1,"filename":"ustawa.pdf","chunk_index":3,"page":12,"preview":"...", "distance":0.24}]

event: token
data: "Konstytucja "

event: token
data: "Rzeczypospolitej "

event: done
data: {}

```

6. Obsługa modeli LLM

Aplikacja współpracuje z dowolnym modelem dostępnym przez Ollama. Poniżej lista wstępnie skonfigurowanych modeli wraz z charakterystyką.

Model	Rozmiar	Charakterystyka
llama3.2:3b	2.0 GB	Domyslny. Szybki, dobry model ogólny. Działa na CPU.

qwen2.5:1.5

Specyfika modeli Bielik (GGUF via hf.co/...)

Modele Bielik pobrane przez Ollama z repozytorium HuggingFace otrzymują szablon czatu Command R (Cohere), mimo że zostały wytrenowane na formacie Mistral [INST]. Powoduje to echo promptu systemowego zamiast odpowiedzi. Aplikacja wykrywa te modele automatycznie i:

- Pomija rolę 'system' (która jest echo-owana przez template Command R).

- Używa /

7. Testy

Projekt zawiera testy end-to-end w pliku `src/test_e2e.py`. Testy używają `httpx.ASGITransport` — nie wymagają uruchomionego serwera ani Ollamy. Wywołania do Ollama są podmieniane deterministycznym mockiem.

Uruchamianie testów

```
# Z katalogu głównego projektu:  
python src/test_e2e.py
```

Co pokrywają testy

- GET `/api/health` — status systemu.

- POST `/a`

Dokumentacja wygenerowana automatycznie.